

01

软件危机：四大本质难题

软件项目管理概念（重要）

软件项目管理的视角

广义软件过程

思考讨论

02

本质难题

软件完全依附于硬件

典型特征

典型软件过程和实践

软件典型过程和实践

方法之一：形式化方法

方法之二：结构化设计和瀑布模型

成熟度模型CMMI

问题

网络化与服务化

迭代式

敏捷 Agile

why 重构

敏捷开发 方法

软件应用典型特征

03

团队动力学

自主团队

在项目进展过程中获得管理层的支持

04

TSP/PSP

TSP课程实践

TSP Launch

典型 TSP 角色

SCRUM

产品负责人职责和工作

开发团队职责和工作

Scrum Master 职责和工作

TSP SCRUM 相似性

估算

SP 和 PROBE

概要设计

SCRUM 中的 Story point

关于估算方法的反思

历史数据的获取——度量

数据治理

工作分解结构（Work Breakdown Structure, WBS）

创建 WBS 方法

过程架构-生命周期模型

通用计划框架

任务计划和日程计划

质量计划原理和方法

风险计划

风险应对

项目跟踪

挣值管理方法 EVM

EVM 三种实现

- EVM 的局限性
- 里程碑评审
- 其他计划跟踪
- 纠偏活动的管理
- 项目总结的意义
- 基于 PMBOK 的总结

05

- 质量管理
 - 质量概念
 - 面向用户的质量观
 - 典型用户质量期望
 - 个人软件过程 (PSP) 质量策略
 - 评审发现缺陷典型流程
 - PSP 评审过程质量
- 质量指标
 - 质量指标之一: Yield
 - 质量指标之二: 质检失效比 A/FR
 - 质量指标之三: 过程质量指标 PQI
 - 质量指标之四: Review Rate
 - 质量指标之五: DRL
- 设计
 - 设计的内容
 - 设计验证方法
 - 状态机验证

06

- 团队工程
 - 需求类别
 - 需求获取
 - 需求汇总
 - 需求验证
 - 优秀需求规格文档特征
 - 团队设计
 - 团队智慧
 - 设计标准
 - 复用性支持
 - 可测试性考虑
 - 实现策略
 - 集成策略
 - 大爆炸集成策略
 - 逐一添加集成策略
 - 集簇集成策略
 - 扁平化集成策略
- 验证与确认

07

- 项目支持活动
 - 配置管理基本概念
 - 度量与分析活动
 - 决策分析
- 根因分析与解决方案
 - 根因分析典型示例

08

- 定量管理仿真建模

期末总结

软件危机：四大本质难题

《人月神话》软件的四大本质困难和挑战： 1. 不可见性：软件项目是一个逻辑实体 2. 复杂性：实体数量众多 3. 可变性：需求变更 4. 一致性：向上向下兼容

软件危机：是指落后的软件生产方式无法满足迅速增长的计算机软件需求，从而导致软件开发与维护过程中出现一系列严重问题的现象

软件工程：是一门研究用工程化方法构建和维护有效的、实用的和高质量的软件的学科。

软件工程的两大视角： 1. 管理视角——能否复制成功？ 2. 技术视角——是否可以将问题解决得更好？

软件项目管理概念（重要）

管理的三大关键要素：目标、状态：是在接近目标还是在远离目标、纠偏。

大部分情况下，管理仅仅是试图复制其他地方（场景）的成功，但是这种复制一般不容易。

软件项目管理是为了降低/减少各种无谓的损耗来实现本该有的性能。

软件过程改进是为了达到更好的效能，其中质量/缺陷是首要目标或限制。

软件项目管理： 1. 典型的三大目标：成本、质量、工期； 2. 软件项目管理是应用方法、工具、技术以及人员能力来完成软件项目，实现项目目标的过程；具体包括：估算、计划、跟踪、风险管理、范围管理、人员管理、沟通管理，等等

软件项目管理的视角

成功是否可以复制？

软件项目管理的两种视角：

1. 软件过程：软件过程是为了实现一个或者多个事先定义的目标而建立起来的一组实践的集合。这组实践之间往往有一定的先后顺序，作为一个整体来实现事先定义的一个或者多个目标。

软件过程 - 软件开发流程图，承载优秀开发经验的载体

2. 生命周期模型：对软件过程的一种人为的划分，对软件过程特征的大致描述

广义软件过程



1. 理论基石：软件产品和服务的质量，很大程度上取决于生产和维护该软件或者服务的过程的质量。GQM之前，传统生产制造业在生产最终通过质量检测进行把关。
2. GQM 目标-问题-指标 **注重过程** [GQM 概述：构建研发效能度量体系的根本方法 - 知乎 \(zhihu.com\)](https://zhuanlan.zhihu.com/p/100000000)
3. 广义软件过程包括技术、人员以及**狭义过程**
4. 仅包含过程的是狭义软件过程
5. 广义软件过程的同义词：软件开发方法、软件开发过程
 1. 净室 Cleanroom 方法、极限编程方法、SCRUM 方法、Gate 方法；Cleanroom 工程过程和 CMM 管理过程互为补充；Cleanroom 比 CMM 更注重质量，更偏向于使用一些数学工具
 2. 而更一般的，敏捷软件过程/方法、轻量型过程/方法以及重型过程/方法等描述也是恰当的
6. 工程过程组 进行软件过程的维护升级改造优化 EPG
7. **敏捷XP**是软件开发方法 CMM是软件过程管理方法
8. 软件过程管理和软件过程改进 含义相近 过程改进包含于过程管理
元模型：必须仿照的基础
9. 虽然基于不同的模型，但最终具体方案均有差异

思考讨论

1. 软件项目管理 - 项目如何实现；软件过程管理 - 流水线升级改造
2. 敏捷过程 也算是一种开发过程 敏捷开发反对成熟度模型 - 敏捷开发将有国际标准？
3. cmm不是软件开发方法
4. cmm是过程管理方法 - 不是软件项目管理方法
5. PDCA，IDEAL是软件过程改进参考元模型，因此是适合在敏捷环境中使用的，在任何软件过程开发过程中均可作为参考
6. 生命周期模型是由人类划分的

本质难题

软件开发有很多困难，但是本质难题是：

1. 不可见性：软件项目是一个逻辑实体
2. 复杂性：实体数量众多
3. 可变性：需求不断变化
4. 一致性：向上向下兼容

进一步分析：

1. 除了不可见性以外，其他三个本质难题因项目而异。
2. 四大本质难题互相促进。
3. 本质难题变化带动软件方法（过程）演变。

本质难题无法彻底解决。

软件完全依附于硬件

典型特征

1. 软件应用典型特征：软件支持硬件完成计算任务、功能单一、复杂度有限、几乎不需要需求变更。
2. 软件开发典型特征：硬件太贵、团队以硬件工程师和数学家为主。

典型软件过程和实践

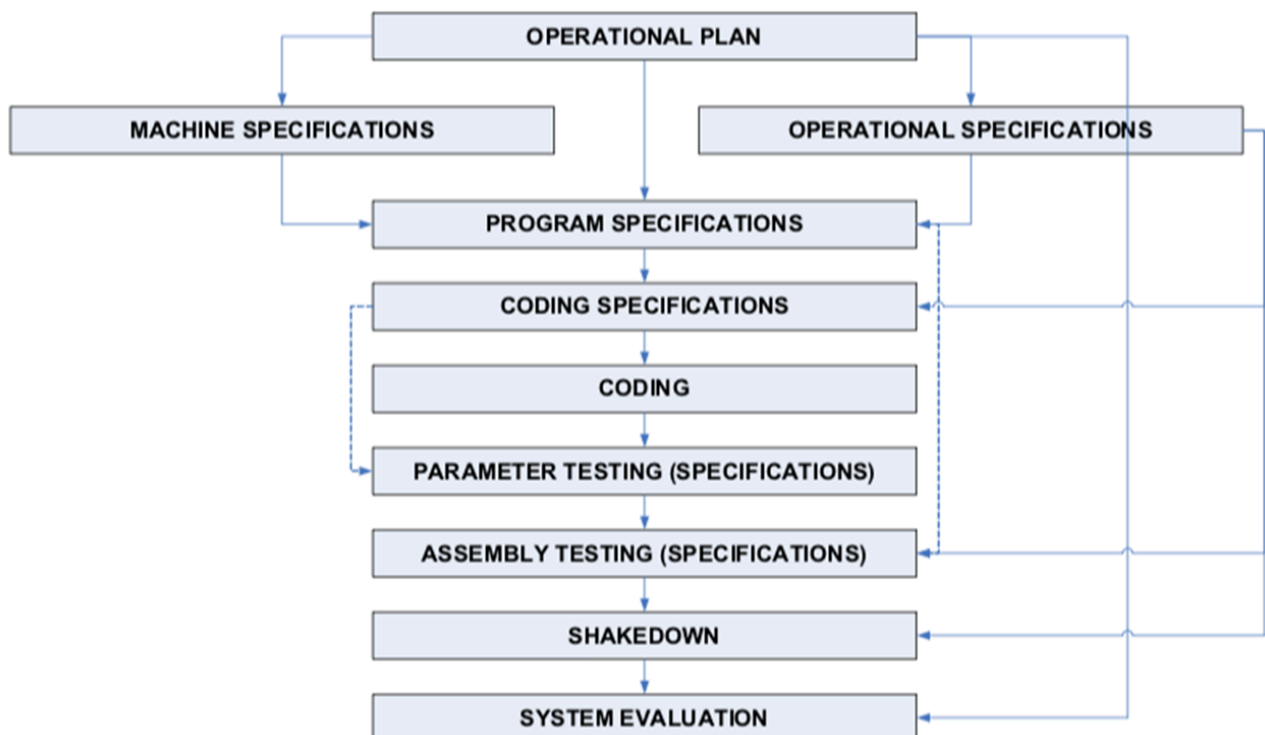


Fig 5 The software development process for SAGE which is similar to hardware development process

1. Measure twice, cut once：这句话强调了在应用更改之前仔细检查代码以在问题成为主要问题之前识别和纠正问题的重要性，在当今时代，**代码的修改需要经过评审、阅读。人读过两次后才能入库？**

2. 相同的软件工程实践：code review & inspection
3. 整个过程非常线性
4. 全部输入输出事先完全确定不会修改

软件典型过程和实践

方法之一：形式化方法

将所有的方法当作数学方法，做数学化的检验，主要解决质量和正确性问题。

主要只能运用在开发团队侧，用户无法参与，且团队内也难以理解

方法之二：结构化设计和瀑布模型

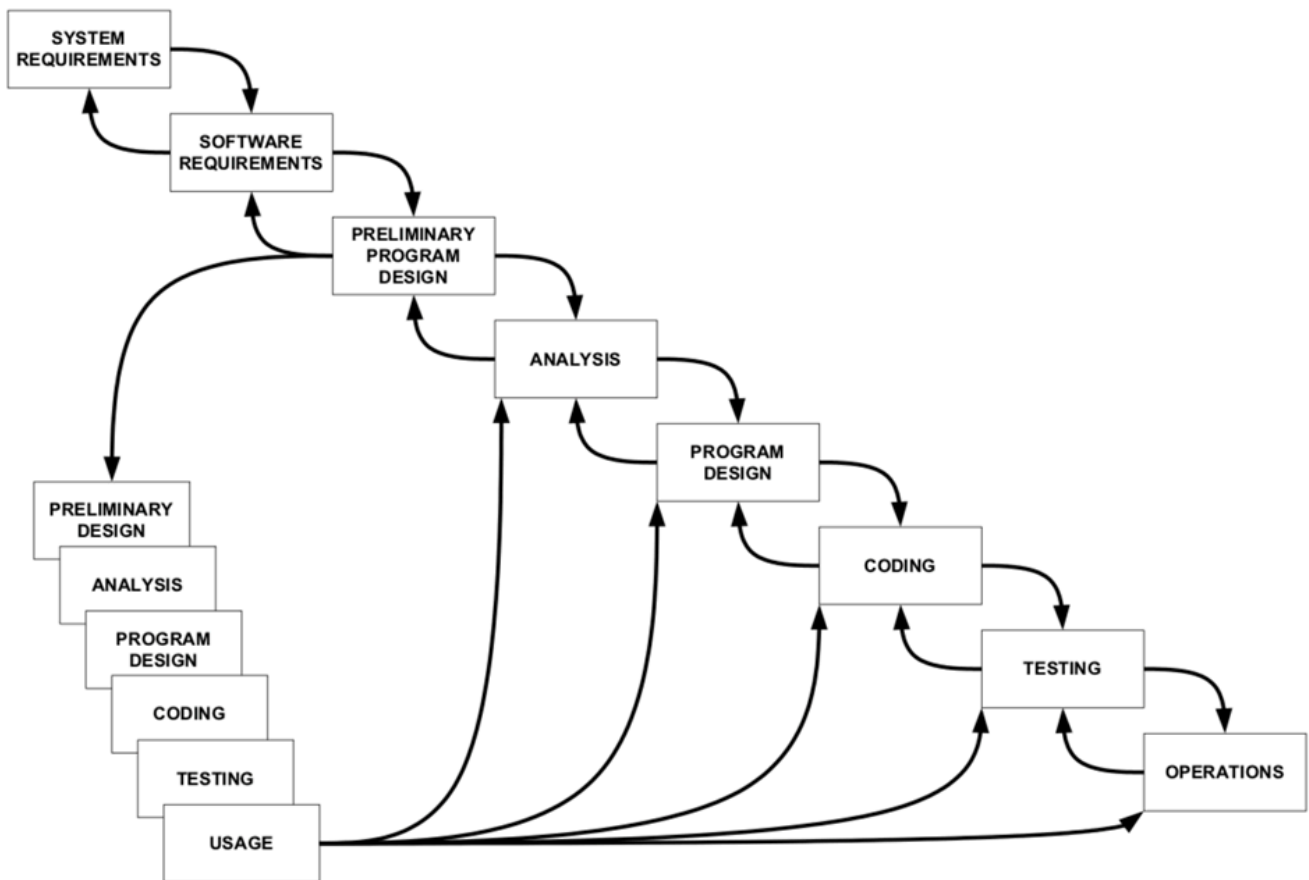
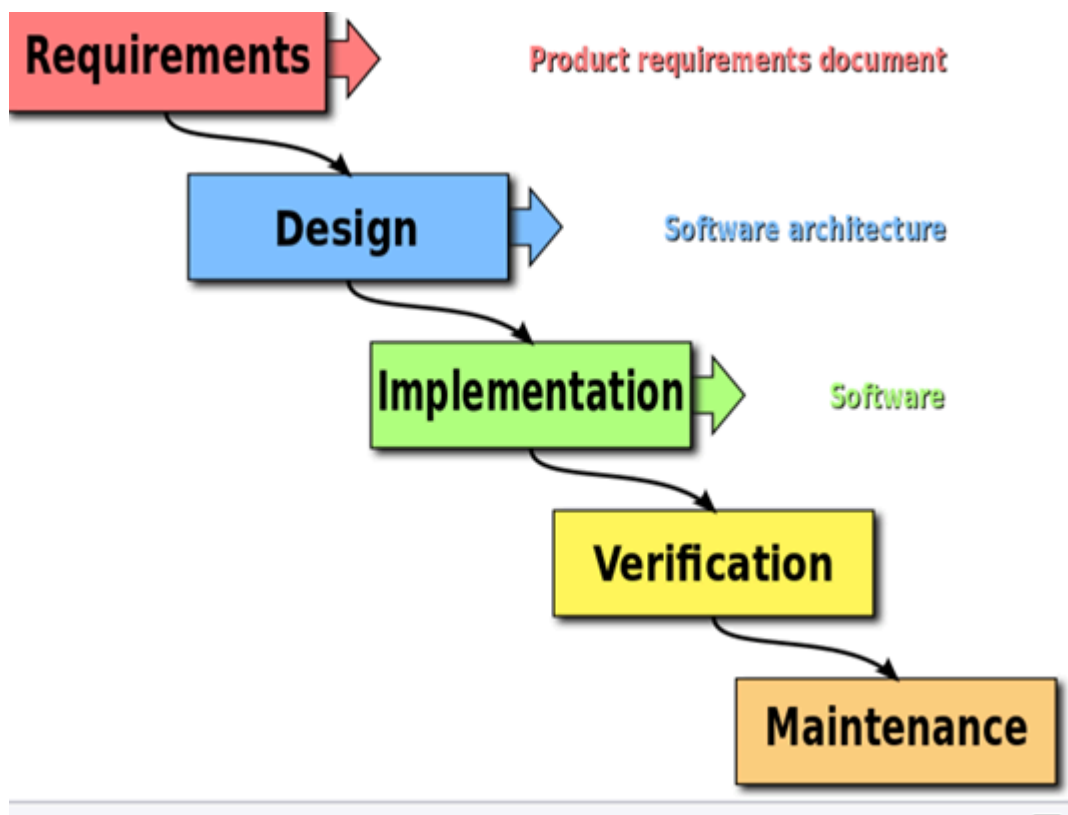


Fig 6 The Royce Waterfall Model

1. 上图才是正确的瀑布模型
2. 不是一个单一模型，是很多的元素的融合，是**系列**的生命周期模型，包括编码改错、改进等等。与迭代并不矛盾。
3. 软件项目团队**应根据软件开发的实际情况进行调整**，下图是被简化过的被美国军方暂时使用过的版本



上图是瀑布模型，每个阶段都有一个往回的箭头；下图不是生命周期模型。

1. 自顶向下，逐步求精。
2. 问题和不足（效率和质量）：
 1. 形式化在扩展性和可用性方面存在不足。
 2. 瀑布模型成为一个重文档、慢节奏的过程。

成熟度模型CMMI

软件过程改进的参考模型

将170+条软件工程最佳实践划分为5个等级，用于给公司的软件开发过程打分，判断公司处于哪一个等级

CMMI 评估**企业在实现自身定义的商业目标上的能力** 商业能力是对外竞争的能力 CMMI 可以评估 商业目标接近的公司中 哪家公司实现目标的能力最强

等级：

1. 初始状态，黑盒不可预测结果、盲目开发、救火文化
2. 项目小组管理，每个项目小组有一套可重复的方法，可被重复成功
3. 公司层面标准化流程，（whyCMMI3? 降低经验教训在组织内的传播成本）裁剪规范-定义小组特性 设定里程碑
4. 定量管理，预测模型（需要大量数据，难以预测偶发现象，假设正常）

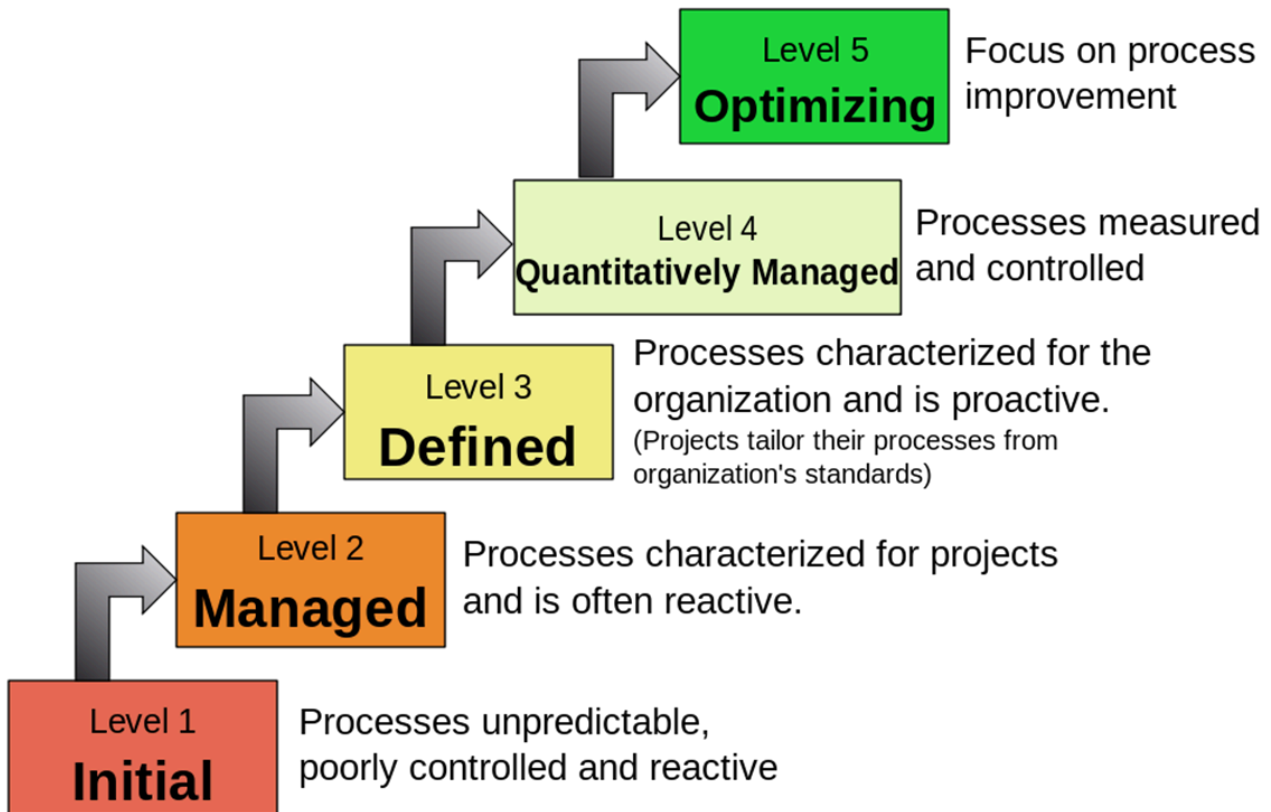
对未来结果进行预测（汽车导航软件实时预测到达时间 根据交通情况更改预计到达时间）

过程控制-让过程的上限下限尽量接近 使得结果可预期

1-3是定性的粗粒度的管理

5. 通过引入新方法 改进预测的期望值，持续改进

Characteristics of the Maturity levels



问题

CMMI 不是过程优劣的标准，也不适合用作公司之间的能力比较？

CMMI 本身是有评级。美国国防部订单招标要求企业至少达到 CMMI 的 3 级。CMMI 评估**企业在实现自身定义的商业目标上的能力** 商业能力是对外竞争的能力 CMMI 可以评估 商业目标接近的公司中 哪家公司实现目标的能力最强。因此CMMI 不能直接评估公司开发能力强弱

网络化与服务化

软件应用特征：功能更复杂，规模更大、用户数量急剧增加（这会带来什么问题？） 、快速演化和需求不确定、分发方式的变化（SaaS软件即服务） 研发到上线时间间隔极短，对交付的质量要求高

迭代式

迭代式：大型软件系统的开发过程也是一个逐步学习和交流的过程，软件系统的交付不是一次完成，而是通过多个迭代周期，逐步来完成交付

用户与开发人员的学习交流主动降低了需求变更的幅度与风险，例如微信以前版本的发布改变了用户的使用习惯，用户和开发者逐渐趋向同一个需求方向

开源软件 - 用户与开发者是同一个群体 共同学习 需求高度一致

敏捷 Agile

都是敏捷方法：FDD DSDM LD RUP

雪鸟会议和敏捷宣言

1. 个体和互动胜过流程和工具
2. 可以工作的软件胜过详尽的文档
3. 客户合作胜过合同谈判
4. 响应变化胜过遵循计划 - 拥抱变更：基于迭代开发的共同学习
5. 也就是说，**尽管右项有其价值，我们更重视左项的价值。**

在右项的基础上进一步发挥左项的价值

why 重构

减轻技术负债

敏捷开发 方法

1. XP (eXtreme Programing) 极限编程 方法：偏重于一些工程实践的描述、

例如：TDD测试驱动开发 先写测试用例 根据测试用例写代码：

1. 倒逼严谨地写测试用例 增加代码稳定性
2. 先写测试用例的过程加深了对代码框架与需求的理解

期望每一个单元的任务都能够在40h（1周）内完成

结对编程 - 人机结对 - 人人结对：精髓是交换，团队内部频繁交换结对，实现所有人对所有代码的了解 在此基础上才可能实现代码共同拥有

代码共同拥有 - 代码所有人都能改

2. SCRUM：管理框架和管理实践 每次冲刺（一个开发周期，2-4周）结束后 都有一个反思的过程 - 用户和开发者共同学习

在一个开发周期内，对于用户需求变更只记录不开发

3. Kanban：

1. 精益生产（丰田制造法）的具体实现 - 生产流水线概念 强调动态 需要流水线中每个小任务都足够小 周期短
2. 可视化工作流、限定 WIP、管理周期时间
3. 马丁提出了微服务架构

4. 开源软件开发方法：是一种基于**并行开发模式**的软件开发的组织与管理方式

1. Linus 定律：如果有足够多的 beta 测试者和合作开发者，几乎所有问题都会很快显现，然后自然有人会把它解决。

Linus定律的时代已经过去，现在个人(低知名度)提交开源代码几乎无人问津

2. 早发布，常发布，倾听用户的反馈、把你的用户当成开发合作者对待，如果想让代码质量快速提升并有效排错，这是最省心的途径、设计上的完美不是没有东西可以再加，而是没有东西可以再减
3. 代码管理：严格的代码提交社区审核制度 项目管理委员会 - 主要进行代码评审
4. 内部开源（inner source） - 早期：只是为了交付 公司内部：你的工具我修改之后为我所用 修改之后的工具再还给你 掌握迭代式开发的精髓
5. 众包（Crowdsourcing）利用互联网将共作发放出去 不存在共同学习关系 只是雇佣关系 只是迭代式开发的形式

软件应用典型特征

1. 进一步服务化和网络化（移动是主流）、随处可见（pervasive）
2. 用户需求多样化进一步凸显
3. 软件产品和服务的地位变化
4. 错综复杂的部署环境
5. 近乎苛刻的用户期望
 1. 多：功能丰富，个性化
 2. 快：快速使用，及时更新，快速解决问题
 3. 好：稳定，可靠，安全，可信
 4. 省：用户的获得成本低，最好免费

团队动力学

自主团队

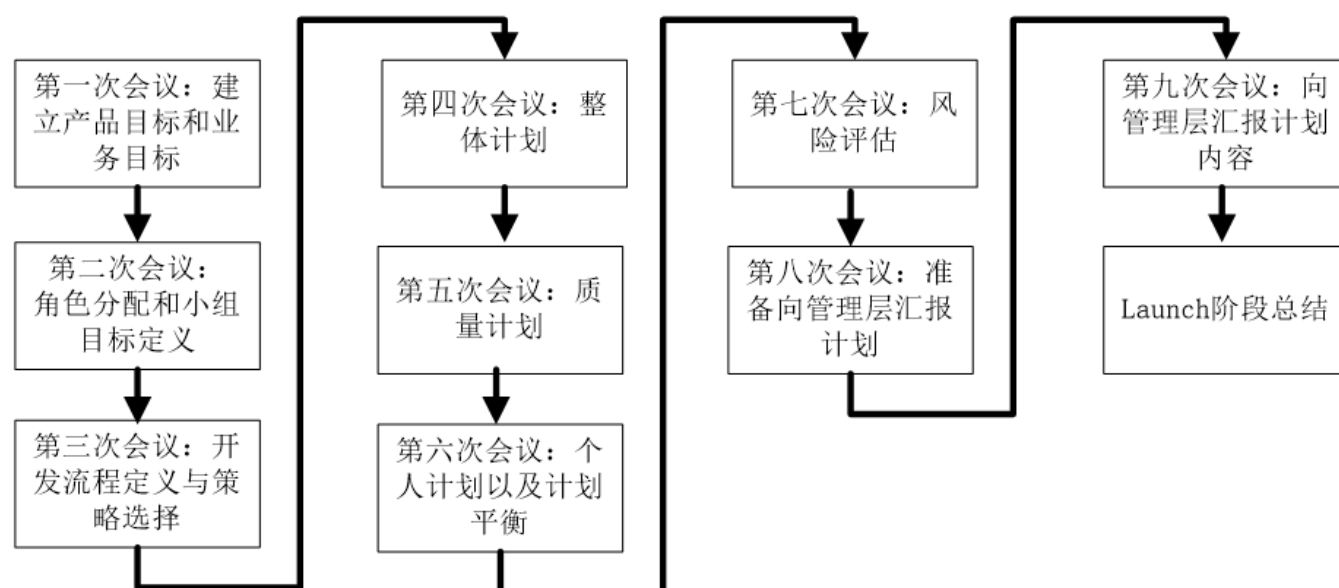
自主团队 和 鼓励承诺 有相似的内涵

在项目进展过程中获得管理层的支持

1. 项目小组应当严格遵循定义好的开发过程开展项目开发工作。通过周例会，进度报告等即时向甲方反馈进度，增进信任
2. 项目小组应当维护和更新项目成员的个人计划和团队计划。
3. 项目小组应当对产品质量进行管理。
4. 项目小组应当跟踪项目进展，并定期向管理层报告。主动向管理层汇报而不是被动等待管理层问询可以最大程度保存小组自主性
5. 项目小组应当持续地向管理层展现优异的项目表现。
6. 高成熟度的质量管理和低成熟度的质量管理有什么区别？差别在于认定偏差的方式不一样：
 1. 2 级、3 级是基于当前已经发生的事实来认定偏差
 2. 4 级、5 级是基于模型来预测最终结果，最后根据这个结果来纠正偏差。

TSP课程实践

TSP Launch



1. 第一次会议：确定产品目标，到底要做什么；业务目标，非功能的要求，例如质量/成本/工期，要做什么；由于资源不足，要确认最后的退让不会影响管理层的核心诉求，并且可以获得尽可能多的资源
2. 第二次会议：角色分配/小组目标 - 每个团队成员/小组的目标：技能锻炼/平衡家庭？需要加以罗列讨论 有目标就要有人盯着跟进 角色分配：开发者/监工/质量 - 每个角色背后都是职责
3. 第三次会议：开发流程- 使用什么样的过程 / 开发策略 - 几次迭代？每次迭代做什么？每次迭代周期的步骤是可以动态调整的，例如上一个周期设计很完善这一个周期可以直接开始编码沿用上一个迭代的设计；需求优先级和业务目标直接相关；项目中每个组件应该如何获取：修改？白嫖？自己写？每个迭代中的复用目标/复用率
4. 第四次会议，整体计划：开发过程中需要做哪些事？
 1. 产出物与规模的确认
 2. 代码评审/设计文档/... 的用时
 3. 做这些事所需的资源 够不够
5. 第五次会议，质量计划：打算做哪些质量实践，到什么程度 做了测试和评审不一定就能实现质量目标
6. 第六次会议，个人计划 - 依据每个人的生产效率；计划平衡：确定每个阶段完成的时间，用微调将每个人参差不齐的完成时间对齐是鼓舞人心的 找到最快完成时间 代码评审不是每个人都需要参加/调整开发内容
7. 第七次会议，风险评估：需求变更/人员配置/人员变动/技术难题 - 影响假设条件不成立的因素是风险 未来的不确定事件对开发造成正面影响 - 机会 也要纳入风险评估的讨论当中 - 如何抓住机会？应对风险？
8. 第八次会议，准备汇报：前面会议的汇总 资源不够时 体现团队诉求 体现深思熟虑的诚意 整个计划的过程/开了几次会/会议记录 应对甲方的质询
9. 第九次会议，定下协议

典型 TSP 角色

1. 项目组长：团队领导者

1. 激励团队成员积极工作，合理处理团队成员的问题，向**管理层**提供项目进度相关的完整信息，分配工作任务
2. 合格的会议组织者和协调者，主持例会
3. 有能力识别问题关键并做出客观决策 项目宏观认知 产品经理对最终业务目标认知和深刻 - 更经常作为决策者/团队领导者

2. 计划经理：开发完整的、准确的团队计划和个人计划；每周准确的**报告项目小组状态给团队领导者** 跟踪和度量组员的工作 第四次和第六次会议的组织者

1. 做事有条理和逻辑 有条理的计划
2. 对于过程数据非常敏感，善于通过每周输入的数据来了解项目当前状况

3. 开发经理：开发优秀的软件产品；充分利用团队成员的技能；**开发技能最强** 制定开发策略 产品规格说明 产品规模估算与资源估算

1. 具备足够的背景可以胜任设计师的工作，并且可以领导设计团队开展工作
2. 熟悉主流的设计工具

4. 质量经理：项目团队严格按照质量计划开展工作，开发出高质量的软件产品；所有的小组评审工作都正常开展，并且都形成了评审报告；向项目组长警示质量问题 - 统计学家 通过数据进行预测；代码评审的组织者

1. 有评审方面的经验，熟悉各种评审方法
2. 关注产品质量

5. 过程经理：所有团队成员准确的记录、报告和跟踪过程数据；所有的团队会议都有相应会议记录。每个团队成员严格按照过程执行

1. 开发过程正确执行

6. 支持经理：项目小组在整个开发过程中都有**合适的工具和环境**；对于基线产品，不存在非授权的变更；项目小组的风险和问题得到跟踪；项目小组在开发过程中满足复用目标

1. 熟悉开发环境的不同版本 以及配置管理流程
2. 管理代码仓库
3. 复用率随着迭代周期不断上升 - 有意识地提升复用比例以满足复用目标
4. 主持配置管理委员会 只有得到支持经理认可才能进入生产环境

7. 安全经理？性能经理？

8. 开发人员：团队中每个人员除了是开发人员外还有一个方面的经理角色 每个人都在从一个方面管理着团队

SCRUM

scrum每个迭代时间窗口固定，便于公司管理

每个迭代内只记录需求变更但暂时不响应

每个周期结束例会总结得失

[\(21 封私信 / 80 条消息\) 燃尽图是什么？ - 知乎 \(zhihu.com\)](#)

Scrum 中的文档：

1. 产品订单（product backlog）：整个项目的概要文档，包含了已划分优先等级的、项目要开发的系统或产品的需求清单，是动态的。

2. 冲刺订单 (sprint backlog) : 细化了的文档, 包含了团队如何实现下一个冲刺的需求信息。

1. 哪些产品订单会加入一次冲刺由冲刺计划会议决定。会议中, 产品负责人告诉开发团队他们需要完成产品订单中的哪些订单项, 开发团队决定在下一次冲刺中承诺完成多少订单项。
2. 在冲刺的过程中, 没有人能够变更冲刺订单 (sprint backlog), 这意味着在一个冲刺中需求是被冻结的。

燃尽图 (burn down chart)

典型 SCRUM 团队由一名产品负责人、开发团队和一名 SCRUM Master 组成

产品负责人职责和工作

产品负责人, 代表利益所有者。

产品负责人的职责是将**开发团队开发的产品价值最大化**。

产品负责人是负责管理产品待办列表的唯一负责人。产品待办列表的管理包括: 1. 清晰地表述产品待办列表项; 2. 对产品待办列表项进行排序, 使其最好地实现目标和使命; 3. 优化开发团队所执行工作的价值; 4. 确保产品待办列表对所有人是可见、透明和清晰的, 同时显示 Scrum 团队下一步要做的工作; 以及 5. 确保开发团队对产品待办列表项有足够深的了解。

开发团队职责和工作

1. 负责在每个 Sprint 结束时交付潜在可发布并且“完成”的产品增量。
2. 开发团队由组织组建并得到授权, 团队自己组织和管理他们的工作。开发团队具有下列特点:
 1. 他们是自组织的。没有人 (即使是 Scrum Master) 有权告诉开发团队应该如何把产品待办列表变成潜在可发布的功能增量;
 2. 开发团队是跨职能的团队, 团队作为一个整体, 拥有创建产品增量所需的全部技能;
 3. Scrum 不认可开发团队成员的任何头衔, 不管其承担何种工作 (他们都叫开发人员)。
 4. Scrum 不认可开发团队中所谓的“子团队”, 无论其需要处理的领域是诸如测试、架构、运维或业务分析; 同时,
 5. 开发团队中的每个成员也许有特长和专注的领域, 但是责任属于整个开发团队。

Scrum Master 职责和工作

促进和支持 SCRUM, 帮助每个人理解 SCRUM 理论、实践、规则和价值

SCRUM Master 是一位服务型领导

1. 帮助 SCRUM 团队之外的人了解如何与 SCRUM 团队交互是有益的
2. 改变 SCRUM 团队之外的人与 SCRUM 团队的互动方式来最大化 SCRUM 团队所创造的价值。

Scrum Master 服务于产品负责人

1. 确保 Scrum 团队中的每个人都尽可能地理解目标、范围和产品域;
2. 找到有效管理产品待办列表的技巧;
3. 帮助 Scrum 团队理解为何需要清晰且简明的产品待办列表项;
4. 理解在经验主义的环境中的产品规划;
5. 确保产品负责人懂得如何来安排产品待办列表使其达到最大化价值;
6. 理解并实践敏捷性; 以及,
7. 当被请求或需要时, 引导 Scrum 事件。

Scrum Master 以各种方式服务于组织

1. 带领并作为教练指导组织采纳 Scrum；
2. 在组织范围内规划 Scrum 的实施；
3. 帮助员工和利益攸关者理解并实施 Scrum 和经验导向的产品开发；
4. 引发能够提升 Scrum 团队生产率的改变；以及，
5. 与其他 Scrum Master 一起工作，增强组织中 Scrum 应用的有效性。

TSP SCRUM 相似性

1. scrum master 和 tsp coach 帮助团队理解运用方法
2. 团队成员相对平等 扁平化 tsp每个人都有特定经理只能 相互管理
3. 都鼓励自主团队
4. 迭代式方法

团队磨合：

1. scrum在开始前做一个特别小的功能 获取成功
2. tsp团队一起规划制订方案 进行磨合

估算

1. 估算目的是给各类计划提供决策依据
2. 估算对象（规模、时间和日程）
 1. 规模：功能点 代码行
 2. 时间：人月可以表示一定的资源投入规模 人月互换有欺骗性
3. 估算只给自己团队估算 估算结果都公开
4. 估算的常见困惑
 1. 项目交付日期、团队组成等都确定了，估算的意义在哪里？到底是估算的准还是只给了这么多时间？但是代码规模的估算不容易受到影响，因此**代码规模估算更重要**
 2. 哪种估算方法好？
 3. 哪个估算结果更好（靠谱）？
 4. 究竟该如何做好估算？
5. 估算是主观猜测，估算能力很难提升？使用了COCOMO等估算模型似乎没有本质区别

20行 10min

SP 和 PROBE

1. 规模度量/估算的困境
2. 以 LOC VS. FP 为例
 1. 精确度量方式往往不便于早期规划/估算；
 2. 有助于早期规划/估算的度量往往难以产生精确度量结果；
3. PROBE（PROxy Based Estimation）的作用：精确度量和早期规划之间的桥梁

相对大小矩阵

	小型 (平方 尺)	中等 (平方 尺)	大型 (平方 尺)
卧室	90	140	200
卫生间	25	60	120
厨房	100	130	160
客厅	150	250	400
书房	150	240	340

早期规划的需求：

用途	要求
厨房	1个中等大小
卧室	1个大卧室； 2个小卧室
卫生间	1个中等大小；1个小型
书房	1个中等大小
客厅	1个大客厅

产生出精确度量结果： $130+200+90 \times 2+60+25+240+400 = 1235$ (平方尺)

概要设计

1. 估算的第一步是做出一个概要设计：

1. 概要设计不是真实设计
2. 与已有产品/组件 相关联
3. 定义能够产生期望功能的产品元素
4. 估算你计划构造之物的规模

2. 对项目进行划分：

1. 划分到与已有模块向对应
2. 划分到对估算有足够信心

3. 对于大多数的项目，概要设计都应相对较快地完成：例如，1000LOC 以内程序，试着将概要设计时间限制在 10 到 20 分钟之内

4. 为了做出概要设计，需要确定产品功能，以及产生这些功能所需的程序组件/模块：“如果我有以下这些部件，我可以构造这个产品。”

5. 然后，将这些程序组件/模块与你以前写的程序相比较，估算它们的规模

6. 最后，将程序组件/模块估算综合给出总规模

如果你对产品的理解还不足以产生一个概要设计，那么你所知道的还不足以做出一个计划

第一次会议结束的标准 - 能做概要设计的时候

1. 当估算多个部件时，总的误差会比各个部件误差的总和要小。

1. 误差趋于抵消了
2. 假设没有共同的偏差

2. 估算要点之一：**尽可能划分详细一些**

1. 主观上提升估算信心
2. 高估与低估相互抵消

3. 估算要点之二：建立估算结果信心

4. 估算要点之三：依赖数据

SCRUM 中的 Story point

估算不应直接估算规模结果 - 应当先**估算模块特征** - 中等大小的输入输出模块？

度量实现一个故事（Story）需要付出的工作量 - 故事抽象出若干单位故事点

1. **抽象**：混合了对于开发特性（feature）所要付出的努力、开发复杂度、个中风险以及类似东西
2. **相对**：设定标准之后，考虑其他特性（feature）与标准之间的相对大小关系，**考虑与已有的定下来的目标之间的相对关系** 相对代码行/相对工作量 能够比较进行讨论

关于估算方法的反思

估算的目标：规模估算 VS. 时间估算 - 时间估算不确定性强 但时间估算往往是最有用的 - 制订计划日程安排

估算结果重要吗？估算要点之四：估算要的是过程，而非结果；估算的过程是**相关干系人达成一致共识的过程** 建立执行信心 生产效率更能与之匹配

有效工作时间并不乐观 - 每天真能有效工作8小时？

历史数据的获取——度量

上世纪：外包公司 - 赚人月差价 - 需要准确估算对其人月

世纪初：互联网扩张期 - 圈地运动 - 完成成品的红利完全能够抹平估算不准的浪费

2015年后：互联网大厂降本增效 - 重新开始计较度量

度量体现着决策者对试图要实现的目标的关切程度

数据治理

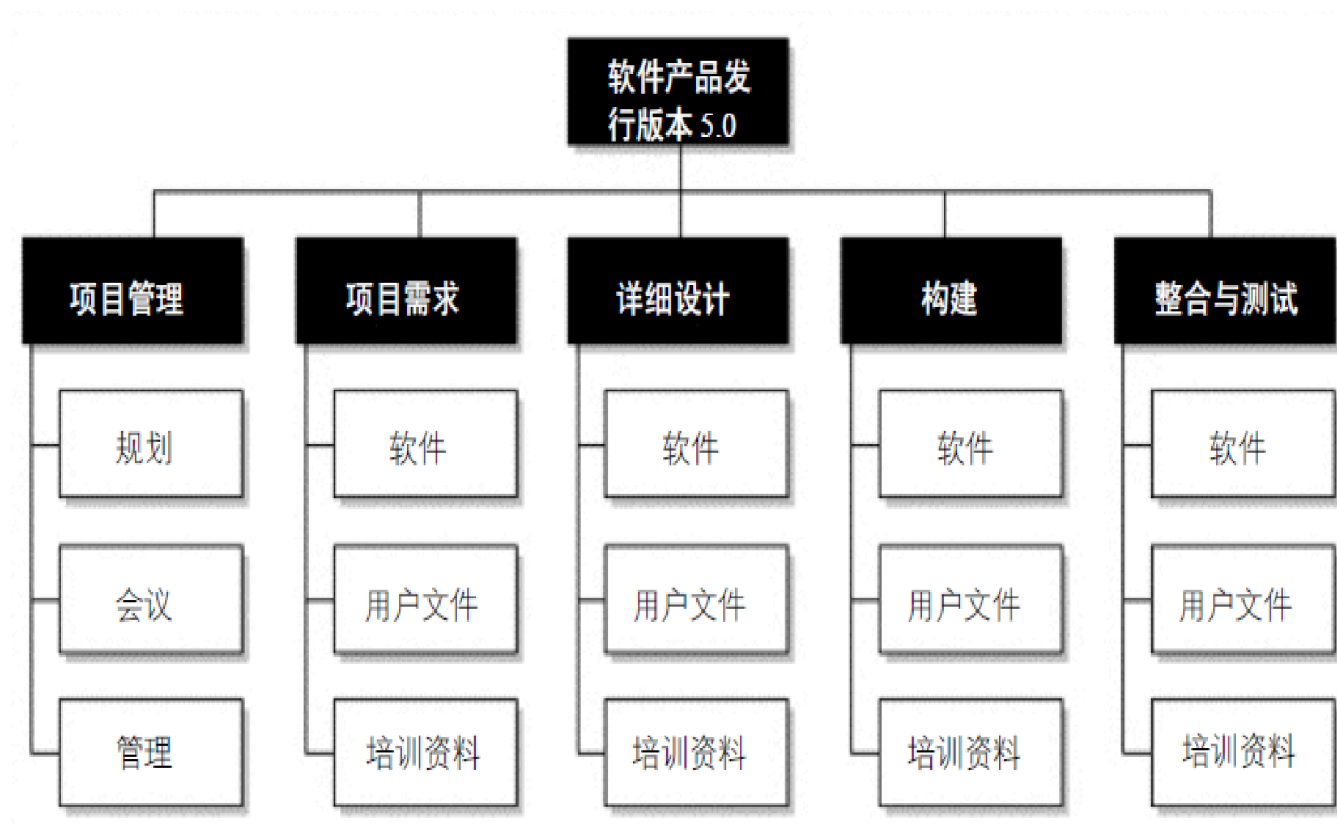
数据的获取 - 统计 - 分析

GQM+：目标-问题-指标（GQM）**自顶向下构建度量指标** 对应 DAMA 软件开发的数字化转型

[研发效能度量体系构建的根本方法—— GQM 从入门到精通 | 思码逸博客 \(merico.cn\)](#)

LLM生成代码关键指标 - 代码采纳率

工作分解结构 (Work Breakdown Structure, WBS)



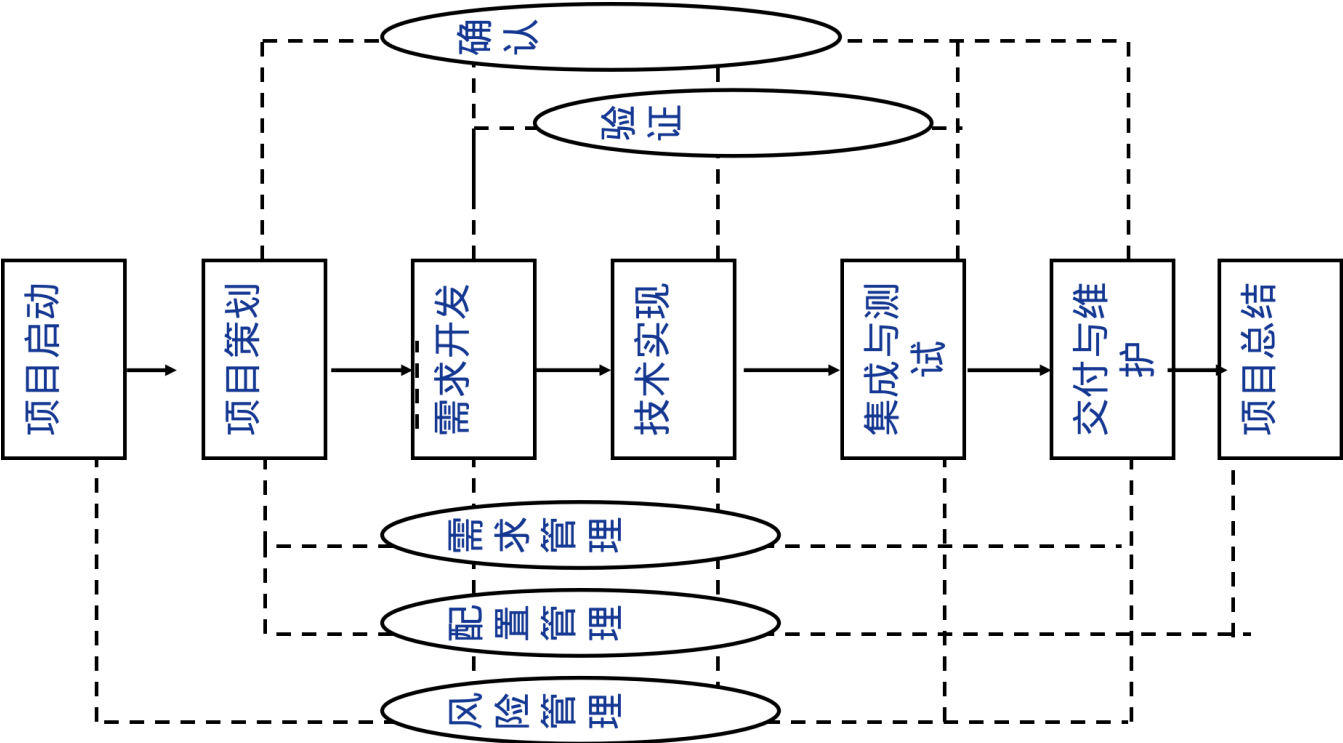
建立项目的整体观

1. 提供整体观
2. 不遗漏可交付物
3. 明确各个角色的责任
4. 工作包定义
5. 估算和计划的基础
6. 理解工作，分析风险

创建 WBS 方法

1. 识别和分析可交付成果及相关工作；
2. 确定工作分解结构的结构与编排方法 - 按（过程/功能）同层内两个标准不能同时用
3. 自上而下逐层细化分解；7+-2
4. 为工作分解结构组成部分制定和分配标志编码；
5. 核实工作分解的程度是必要且充分的。

过程架构-生命周期模型

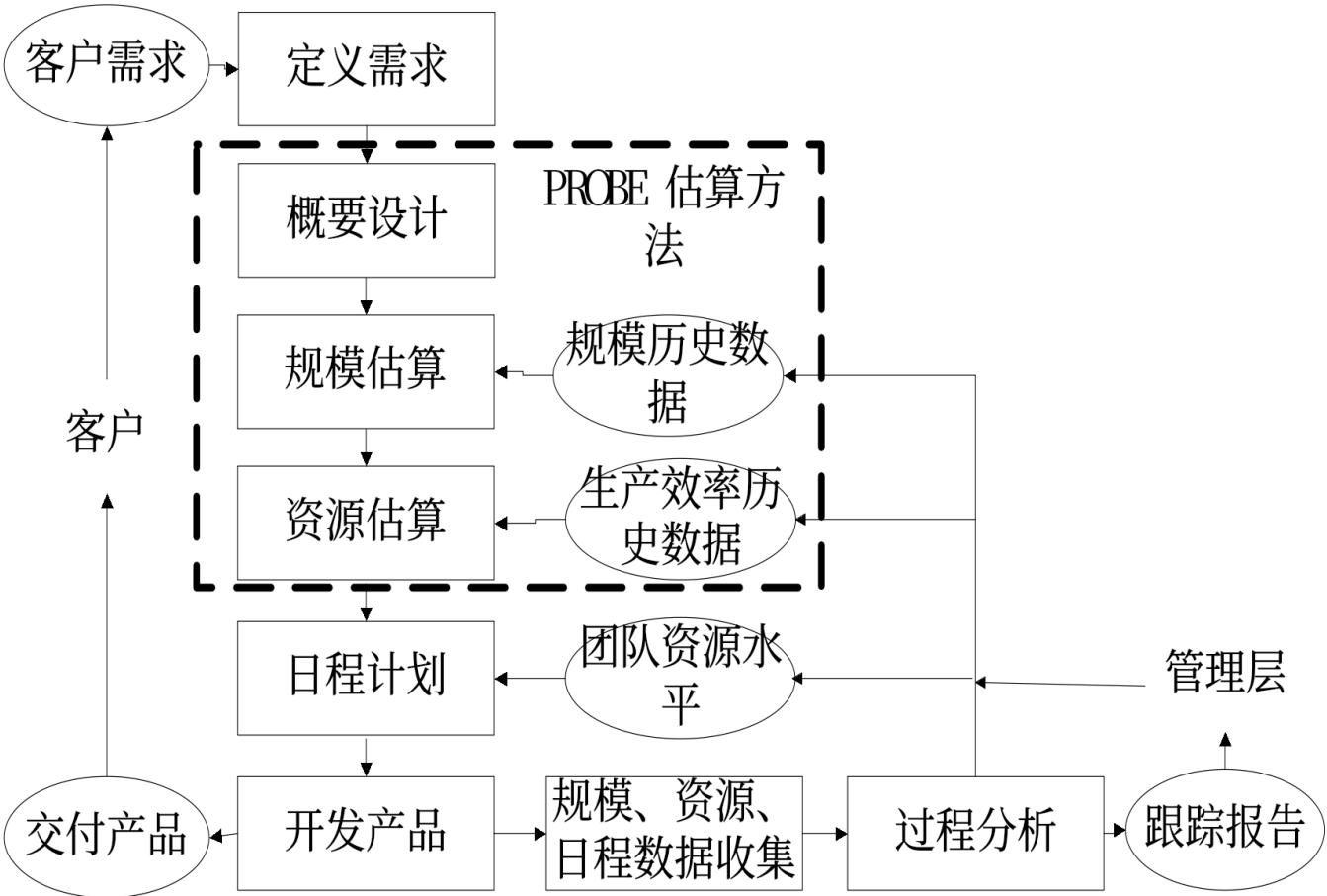


项目策划 需要估算 估算也依赖需求 - 项目启动之前的项目立项就已在进行估算 是否矛盾？

能够完成概要设计后即可估算 - 但到需求开发阶段还需要修改

软件过程应当细节 上图是软件生命周期模型

通用计划框架



对计划进行质询攻防 讨论计划合理性

- 1. 估算资源时间是否能继续压缩 - 如何防御：避免纠缠具体数值 谈论对功能模块体量的认知 最终数据是“计算”出来的
- 2. 团队资源水平如何呈现

任务计划和日程计划

任务清单与资料清单 - 任务耗时与累计时间

任务	需要时间资源（小时）	累计时间资源（小时）
A	2	2
B	3	5
C	3	8

质量计划原理和方法

- 1. 项目的质量计划中应当确定需要开展的质量保证活动。
- 2. 典型的质量保证活动包括个人评审、团队评审、单元测试、集成测试、系统测试以及验收测试等。
- 3. 在质量计划中需要解决的关键的问题是开展哪些活动，以及这些活动开展的程度，如时间、人数和目标分别是什么。
- 4. 打算做哪些**评审**：需求评审，代码评审 - 打算做哪些**测试**：单元测试，集成测试，试运行 - 打算做到**什么程度**：只看理解，看再讨论，通过指标体现测试程度

风险计划

- 1. 风险管理的目的是在风险发生前，识别出潜在的问题，以便在产品或项目的生命周期中规划和实施风险管理活动，以消除潜在问题对项目产生的负面影响。
- 2. 风险管理大致分成两部分，即风险识别和风险应对。
- 3. 风险识别：头脑风暴 = 无法管理/没有系统化
- 4. 风险：[P（可能性），I（影响程度），T（阈值）]
- 5. 可能性高影响程度低：团队内部管理 - 可能性低影响度高：尽量推给团队外处理

风险应对

识别风险之后，就应当制定相应的风险管理策略，以应对各类风险。

典型的策略包括：

- 1. 风险转嫁 - 付钱让别人完成部分技术难题
- 2. 风险解决
- 3. 风险缓解 - 添加人员/提前加班

项目跟踪

开展跟踪活动的目的在于了解项目进度，以便在项目实际进展与计划产生严重偏离时，可采取适当的纠正措施

挣值管理方法 EVM

项目的挣值管理方法（Earned Value Management，简称 EVM）是用来客观度量项目进度的一种项目管理方法。

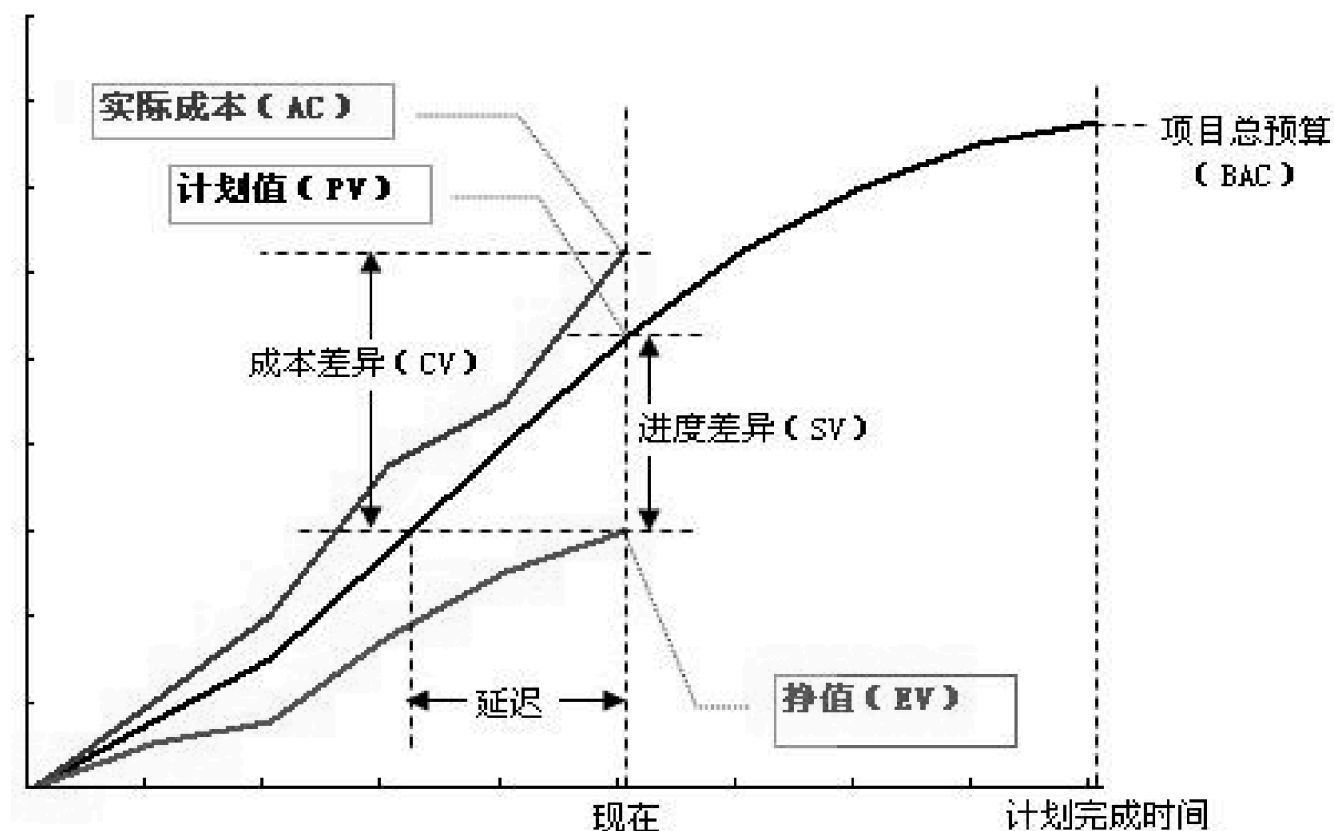
1. 每项任务实现附以一定价值（credit）
2. 100% 完成该项任务，就获得相应价值，只有**完全完成的任务才被认可**

EVM 采用与进度计划、成本预算和实际成本相联系三个独立的变量，进行项目绩效测量

这套方法直接应用到软件项目中会存在问题：将计划-设计-编码-测试进行细分，或者再将编码等一件事按50%规则拆成两份 - 及时获取项目进展可以维持激励水平

在软件项目中，计划投入的时间作为挣值比较合适

EVM 三种实现



1. 上面的线是为了获取这些挣值付出的实际代价，这个线和挣值之间的差异是成本差异。**上图：进度不及，成本超支，成本超支表明正在加班，加班也救不回来**
2. 中间的线是预算（每天需要完成多少挣值）BAC，理想情况下是一条直线。
3. 下面的线是挣值（实际的进展情况）（EV），和 owner value 有关，对应 plan value
4. 实际获取挣值和预计获取挣值的差异是进度差异。

简单实现：这种方式仅仅关注进度信息，**只关注PV和EV之间的关系**。在实现时，首先需要建立 WBS，定义工作范围；其次为 WBS 中每一项工作定义一个价值（PV）；最后按照一定的规则将某一数值赋给已经完成的工作或者正在进行的工作。常用规则分别为 0-100 规则和 50-50 规则，前者只有当某项任务完成时，该任务的 PV 值将转化成 EV 值；后者只需要开始某项任务，即可以赋原 PV 值的 50% 作为 EV 值，完成时，再加上另外的 50%。而实际完成的工作所需成本 AC 不对 EV 值产生任何影响。

中级实现在简单实现的基础上，**加入日程偏差的计算，进行决策与预测**。典型计算方式有：

- 日程偏差 $SV = EV - PV$;
- 日程偏差指数 $SPI = EV/PV$;

- 当挣值落后的时候，除了提升效率追赶进度外，还可以砍掉未完成的部分模块，从而提升已完成的部分的挣值占比

高级实现在中级实现的基础上，还需要考察项目的实际成本。再加入一个**实际成本线AC**

燃尽图是EVM简单实现的变形

EVM 的局限性

1. EVM 这种方式也有一定的局限性：
 1. EVM 一般不能应用软件项目的质量管理。
 2. EVM 需要量化的管理机制，这就使其在一些探索型项目以及常用的敏捷开发方法中的应用受到限制
2. EVM 完全**依赖项目的准确估算（价值体系）**，然而在项目早期，很难对项目进行非常准确的估算，不是说估不准吗？ - 并不矛盾：在共识的前提下，即使出现矛盾大家一起承担也问题不大

里程碑评审

里程碑会议 - 正式 高级管理者和客户都会出席；周会 - 小组内部自己的事

正式会议：回答 项目是否应该进入下一个阶段

其他计划跟踪

1. 日程计划跟踪
2. 承诺计划跟踪
3. 风险计划跟踪
4. 数据收集计划跟踪
5. 沟通计划跟踪 - 开发组提出问题后 甲方3个工作日内要给出回复

纠偏活动的管理

1. 典型的纠偏活动包括
 1. 偏差原因分析
 2. 纠偏措施定义
 3. 纠偏措施管理

项目总结的意义

1. 软件项目的特点决定了持续改善对于软件工程师的重要性。
2. Rita Mae Brown 在书中写到的那样“所谓的愚蠢（Insanity）就是反复做同样的事情，但是期望有不同的结果”
3. 提供一个系统化的方式来总结经验教训、防止犯同样的错误、评估项目团队绩效、积累过程数据等。提供给项目团队成员持续学习和改进的机会
4. 一般情况下，项目总结都包括
 1. 准备阶段
 2. 总结阶段
 3. 报告阶段

基于 PMBOK 的总结

范围管理、时间管理、成本管理、质量管理、人力资源管理、沟通管理、风险管理、采购管理和整合管理 9 大知识领域。

范围管理：软件需求管理 - 需求变更是否被识别和处理？

时间管理：成本管理：

1. 估算偏差有多大？
2. 日程计划准确程度如何？
3. 里程碑偏差有多大？
4. 日程计划有什么变更？为什么？

质量管理：有没有办法在前期消除缺陷

人力资源管理：生产效率

沟通管理：相关干系人 - 对项目有影响的人

质量管理

质量概念

1. 软件质量为内外两部分的特性：其外部质量特性面向软件产品的最终用户，其内部质量特性则不直接面向最终用户 - 内部质量特性：项目可拓展性
2. 软件质量为软件产品可以改变世界，使世界更加美好的程度。从用户的角度考察软件质量，用户满意度是最为重要的判断标准 - 用户满意度
3. 软件质量为对人（用户）的价值。这一定义强调了质量的主观性，即对同一款软件而言，不同的用户对其质量有不同的体验 - 存在主观性，影响了管理的策略 - 软件质量与用户群体、调研时间有关

面向用户的质量观

定义质量为满足用户需求的程度：

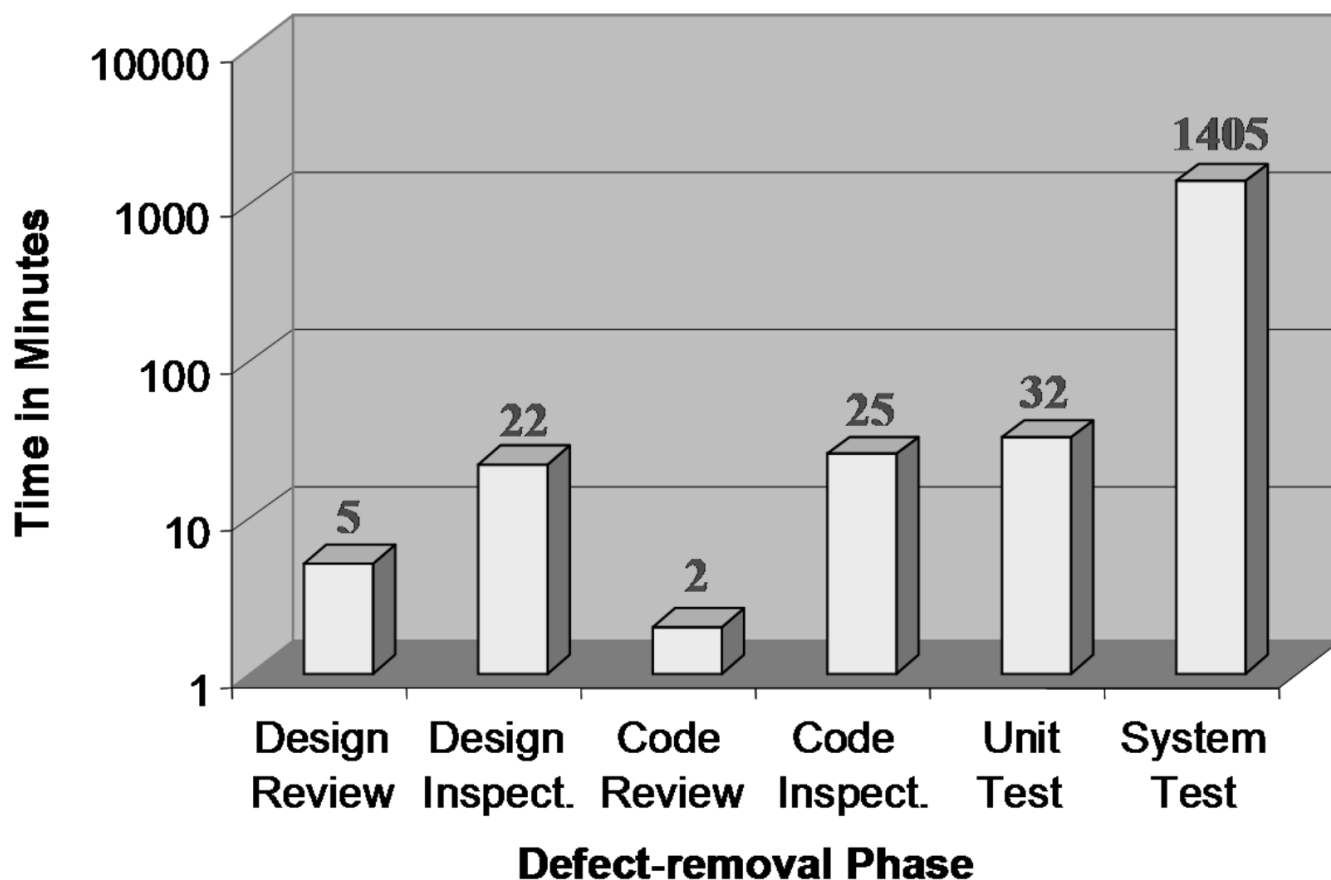
1. 用户究竟是谁？
2. 用户需求的优先级是什么？
3. 这种用户的优先级对软件产品的开发过程产生什么样的影响？
4. 怎样来度量这种质量观下的质量水平？

典型用户质量期望

1. 排第一位：这款软件产品必须能够正确工作

个人软件过程（PSP）质量策略

1. 用缺陷管理来替代质量管理；
 1. bug - 该问题的影响是可预知的
 2. defect - 该问题的影响未知 - 更危险
2. 高质量产品也就意味着要求组成软件产品的各个组件基本无缺陷；
 1. 每个开发者都需要承担更多责任
3. **各个组件的高质量是通过高质量评审来实现的** - 代码评审效率最高 - 但代码评审有门槛
4. 不要将错误拖到系统级再解决 - 尽可能在上游解决
5. 但是并不代表系统测试就不需要，测试也是必要的



评审发现缺陷典型流程

1. 遵循评审者的逻辑来理解程序流程；
2. 发现缺陷的同时，也知道了缺陷的位置和原因 - 评审相对于测试的天然优势
3. 修正缺陷；
4. 重要的缺陷关注：注入阶段、消除阶段和根本原因。

现在大厂研发团队和测试团队分开，测试发现的问题越多越好还是越少越好？

1. 越少越好：测试大约能暴露所有错误的一半 - 测试发现错误越多实则质量越差 - 假设测试团队暴露缺陷的能力相对平稳

PSP 评审过程质量

1. 评审检查表 - 个性化检查表对于新手重要
2. 质量控制指标

质量指标

质量指标之一：Yield

Yield 指标用以度量每个阶段在消除缺陷方面的效率。

1. $\text{Phase Yield} = 100 \times (\text{某阶段发现的缺陷个数}) / (\text{某阶段注入的缺陷个数} + \text{进入该阶段前遗留的缺陷个数})$
2. $\text{Process Yield} = 100 \times (\text{第一次编译前发现的缺陷个数}) / (\text{第一次编译前注入的缺陷个数})$ 是对Phase Yield的汇总

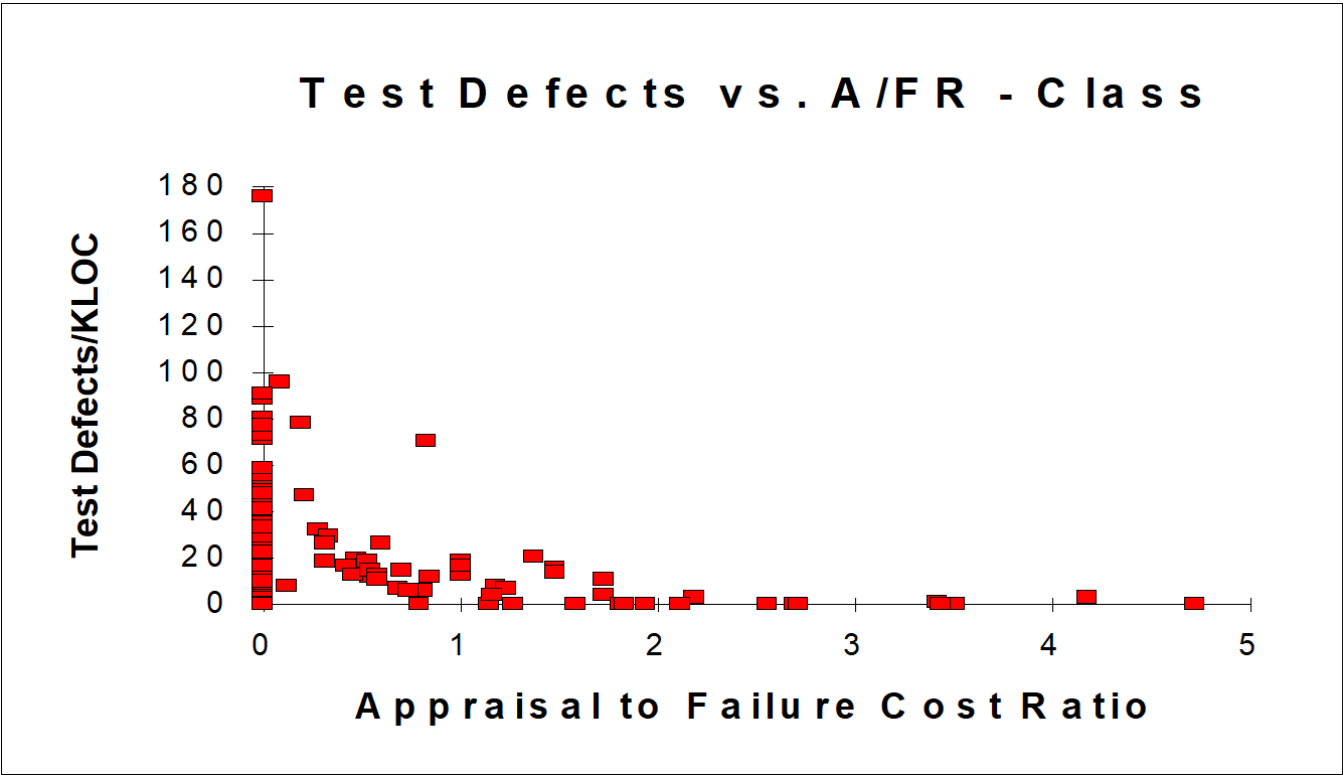
只能预测，不能度量：因为无法得知注入的缺陷总数目。

通过历史数据估算编码阶段注入缺陷总数

蒙特卡洛方法 [蒙特卡罗方法详解 - 知乎\(zhihu.com\)](#)

质量指标之二：质检失效比 A/FR

$A/FR = \text{PSP 质检成本} / \text{PSP 失效成本}$ ，用来测量在第一次编译前花在查找缺陷上的时间的相对值。可用**评审复查时间/（编译+测试）时间**来计算。能很好地指示测试中发现缺陷的可能性。当 $A/FR < 1$ 时，程序测试一般会发现很多错误；当 $A/FR > 2$ 时，过程产生无缺陷的可能性更大。 A/FR 的值对于小的独立的产品通常比 2.0 要大； A/FR 的值对于相对大的产品等于 1.0 较为合适



1. 可以直接通过获取质检失效比来判断项目开发质量
2. 若追求高质量，评审时间应该为测试时间的两倍 - 制订质量计划

质量指标之三：过程质量指标 PQI

5 个数据乘积：0.39

1. 设计质量：设计的时间应该大于编码的时间。 $\min\{\text{设计时间}/\text{编码时间}, 1\}$ 0.416
2. 设计评审质量：设计评审的时间应该大于设计时间的 50%。 $\min\{2 \times \text{设计评审时间}/\text{设计时间}, 1\}$ 0.941
3. 代码评审质量：代码评审时间应该大于编码时间的 50%。 $\min\{2 \times \text{代码评审时间}/\text{编码时间}, 1\}$ 1
4. 代码质量：代码的编译缺陷密度应当小于 10 个/千行。 $\min\{20/(\text{编译缺陷密度}+10), 1\}$
5. 程序质量：代码单元测试缺陷密度应当小于 5 个/千行。 $\min\{10/(\text{单元测试缺陷密度}+5), 1\}$

特性：

1. PQI 越大，表示上面五个“应该”满足地越好，缺陷也就越少。PQI 超过 0.4 即可。
2. 前三个指标是过程质量，后两个指标是结果质量 - 少见地兼顾过程与结果
3. 在CICD之通过PQI评估模块质量
4. 辅助制订质量计划
5. 辅助质量改进建议

质量指标之四：Review Rate

- 1. 评审的速度（Review Rate）是一个用以指导软件工程师开展有效评审的指标
- 2. 高质量的评审需要软件工程师投入足够的时间进行评审
- 3. 在 PSP 的实践中，代码评审速度小于 200 LOC/小时，文档评审速度小于 4 Page/小时
- 4. 200行每小时是最高效的，然而很多初学者都以每小时上千行的速度评审，这是不对的
- 5. code review目标是将所有问题都找出

估算：

- 1. 估算文档数量
- 2. 估算代码数量 - 根据代码行数设定评审时间
- 3. 估算评审时间等等

质量指标之五：DRL

缺陷消除效率比度量的的是不同缺陷消除手段消除缺陷的效率。

其计算方式是以某个测试阶段（一般为单元测试）每小时发现的缺陷数为基础，其他阶段每小时发现缺陷数与该测试阶段每小时发现的缺陷的比值就是 DRL。

单元测试的发现缺陷效率为基准，其效率最低时整体发现缺陷的效率最高

代码编译 - 消除语法错误，编译阶段消除缺陷的效率是最高的

阶段	缺陷个数/小时	DRL (UT)
设计评审	3.6	1.03
编码评审	8	2.29
单元测试	3.5	1

5个指标选几个合适的即可

打印后/能够在一块屏幕上看到方法/模块全貌的评审效果更好

先review再编译能够提升自身技能

先个人评审再小组评审，小组评审的代码应该是高质量的，否则就是在浪费时间

评审9宫图

以比历史水平更快的速度发现了正常的错误数量 - 被评审的文档漏洞很多

比历史水平更慢的速度发现了正常的错误数量 - 评审团队评审技能不熟练

两个人评审同一段代码：

发现的错误完全重合 - 可能一共就这么多错误都发现了

发现错误完全不同 - 错误多得流稀

用缺陷管理替换质量管理

进入测试之前的代码应当就是高质量的

设计

设计的内容

	动态信息	静态信息
外部信息	交互信息（服务、消息等）； 用例图，时序图	功能（继承、类结构等）； 类图
内部信息	行为信息（状态机）； 状态机图	结构信息（属性、业务逻辑、伪代码等）

若这四个方面有缺失，那么设计有缺陷，最终会是一个无法评审的设计

企业要求 - 业务？设计？

设计验证方法

意义：简单评审不足以发现复杂缺陷 - 未来以大语言模型为基础的开发流程中，需求 - 设计 - 代码 的过程中，可以使用**设计验证方法**给予大语言模型生成设计的质量评价，以提升大模型设计的水平

方法：

- 状态机验证
- 符号化执行验证
- 执行表验证
- 跟踪表验证
- 正确性验证

状态机验证

正确状态机：

- 完整：从这个状态触发所有条件都考虑到 - 真值表：竖向上所有列都有内容
- 正交：状态转移唯一 - 真值表：竖向上要么只有一个状态要么两个状态完全相等，才能保证下一个状态唯一

验证方法：

1. 检验状态机，**消除死循环和陷阱状态**。陷阱：进入某个状态之后无法结束
2. 检查状态转换，验证**完整性和正交性**。
3. 评价状态机，检验是否体现设计意图。

n	< nMax				= nMax				> nMax			
Pass word	Valid		!Valid		Valid		!Valid		Valid		!Valid	
ID	Valid	!Valid	Valid	!Valid	Valid	!Valid	Valid	!Valid	Valid	!Valid	Valid	!Valid
Next for Each Defined Transition Condition												
A		CID	CID	CID								
B	End				End				End			
C					End	End	End	End	End	End	End	End
Resulting Values of Fail for Each Defined Transition Condition												
A												
B	F				F				F			
C					T	T	T	T	T	T	T	T

第5列违背正交 - 状态机有问题

团队工程

需求类别

1. 客户需求：对客户问题的描述，成本限制、系统配置等都是客户需求
预算/时间/法律法规
2. 产品需求：针对客户需求提出的解决方案

客户想要做某件事 - 客户为什么要做这件事，客户要解决什么问题？

需求获取

需求诱导：应更加积极地、前瞻性地识别那些客户没有明确提供的额外需求

需求汇总

汇总时解决需求冲突

未显式表述的需求

需求验证

只能评审 - 产品需求能否实现用户需求，解决方案是否有效

1. 建立和维护操作概念和相关的场景

优秀需求规格文档特征

1. 内聚特征
2. 完整特征
3. 一致特征
4. 原子特征
5. 可跟踪特征
6. 非过期特征
7. 可行性特征
8. 非二义性特征
9. 强制特征
10. 可验证特征

团队设计

团队智慧

发挥团队智慧两大挑战：

1. 确定整体架构之前很难进行分工
2. 鼓励团队成员在讨论和评审会议中的参与程度

设计标准

1. 命名规范
2. 接口标准：每个函数参数不超过两个？开发质量高
3. 系统出错信息：标准化出错信息/测试的好处：方便定位错误
4. 设计表示标准

复用性支持

设计阶段必须要充分考虑复用的可能，要主动贴近复用

1. 复用接口标准：在文字中表述接口实现的功能，让别人知道你在做啥，便于复用
2. 复用文档标准
3. 复用质量保证机制

可测试性考虑

先写测试脚本再写业务代码？改善代码结构

实现策略

1. 评审的考虑
2. 复用策略
3. 可测试性考虑

集成策略

大爆炸集成策略

1. 定义：将所有已经完成的组件放在一起进行一次集成
2. 优点：模块质量很高时效率最高(PQI)；需要很少的测试用例
3. 缺点：需要所有有待集成的组件质量非常高，否则会出现难以定位缺陷位置的问题，从而消耗很多测试时间；另外，系统越复杂，规模越大，问题越突出

逐一添加集成策略

1. 与大爆炸集成策略相反，采取一次添加一个组件的方式进行集成
2. 优点：很容易定位缺陷位置，特别是在产品组件质量不高的情况下，每次集成之前都有着坚实的质量基础
3. 缺点：需要测试用例非常多；出现问题修改后存在大量的回归测试，测试时间成本大；因此大型系统开发都会提前预判模块质量，高质量才允许集成

集簇集成策略

1. 定义：是对逐一添加集成策略的改进，把**有相似功能或者有关联**的模块优先进行集成，形成可以工作的组件，然后以组件为单位继续较高层次的集成，自底向上
2. 优点：可以尽早获得一些可以工作的组件，有利于其它组件测试工作的开展
3. 缺点：过于关注个别组件，而缺乏系统的整体观，不能尽早发现系统层面的缺陷，系统级错误最后才会暴露

扁平化集成策略

1. 定义：优先集成高层的部件，然后逐步将各个组件、模块的真正实现加入系统。即尽快构建一个可以工作的扁平化系统。先搭建一个整体框架，自顶向下不断替换桩
2. 优点：可以尽早发现系统层面的缺陷
3. 缺点：为了确保完成的系统，需要大量的打“桩”（stub），即提供一些直接提供返回值的伪实现。这种方式往往不能覆盖整个系统应该处理的多种状态

验证与确认

验证（Verification）和确认（Validation）都是为了提升最终产品的质量而采取的措施。

验证和确认的目的不同：

1. 验证是目的是确保选定的工作产品与事先指定给该**工作产品**的需求一致，确保**解系统内部**的一致性；**产品需求**（解决方案）是否被正确地实现出来 - **验证**
2. 确认的目标则是确保开发完成的产品或者产品组件在即将要使用该产品或者产品组件的**环境中**工作正确，确保**问题域和解系统**的一致性。这个解决方案是否能解决问题(**客户需求**) - **确认**

验证和确认的目的不同，前者关注的是**产品需求**是否得到体现、从这个产品需求（即解决方案）设计出来，一直到解决方案的开发完成，整个过程当中有没有偏、解决方案有没有被正确地实现出来；后者更多关注的是你的这个**客户需求**有没有得到满足，简单来说就是客户期望解决的问题，有没有被解决。

——荣老师复习课

单元测试是验证，CI/CD是验证，需求评审是确认，验收测试是确认。

但实则ver和val贯穿开发始终

项目支持活动

配置管理 - 依赖版本控制，体现开发策略？版本控制是配置管理的最低级别

配置管理基本概念

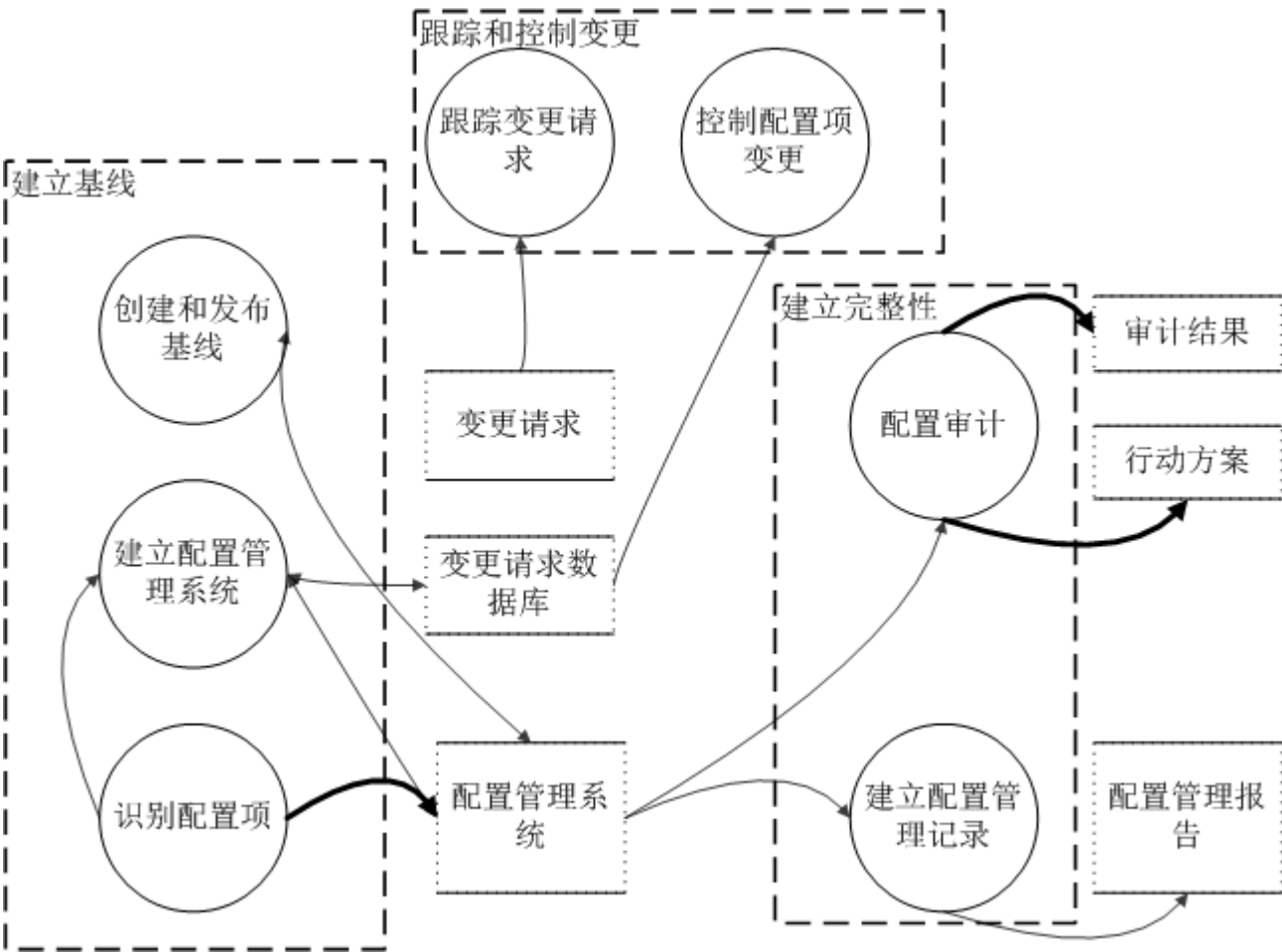
配置项：

- 1. 在配置管理当中作为单独实体进行管理和控制的工作产品集合
- 2. 典型的可能作为配置项纳入配置管理的工作产品包含过程说明文档、项目开发计划文档、需求规格说明书、设计规格说明书、设计图表、产品规格说明书、程序代码、开发环境，如特定版本的编译器等、产品数据文件、产品技术文件、用户支持文档
- 3. 代码一个模块是一个配置项？

基线：基线是一个或多个配置项及相关的标识符的代表，是一组经正式审查同意的规格或工作产品集合，是未来开发工作或交付的基础，而且只能经由严格的变更控制程序才能改变。- 某一个特定时间所有最新版本配置项的组合，里程碑往往和基线同时

- 1. 发布一个基线包括该基线所有的配置项以及这些配置项的最新变更，因此，可以将基线作为接下来工作的基础。
- 2. 典型的发布基线时间点为需求分析之后、设计完成之后、单元测试之后以及最终产品发布。
- 3. 是配置项持续演进的稳定基础

配置管理的目的是建立与维护工作产品的完整性

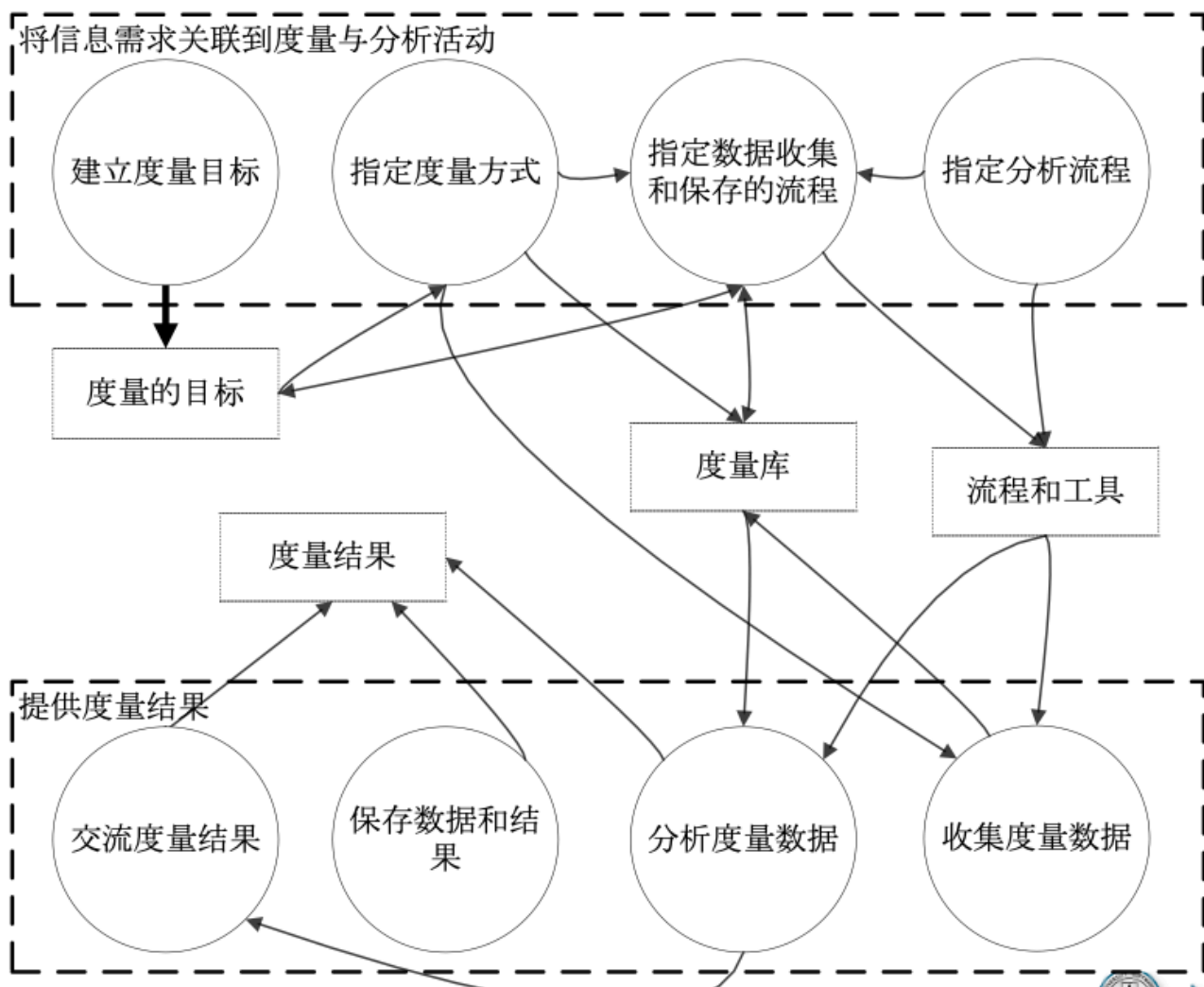


- 1. 识别配置项：做减法，任何配置产物要去掉就导致不完整时即可

2. 建立配置管理系统：不只是配置管理软件，重要是**管控流程**，三库：工作库、配置库（只有少部分人能修改，相当于单个可用的部件）、产品库（完整产品，经过系统测试）； - 回滚：回滚到产品库中； **配置管理 + 管控流程**
3. 跟踪控制变更：单元测试后即认为是稳定版本，修改代码：修改的永远是稳定版本，从配置库中checkout一个稳定版本，不能直接修改工作库
4. 变更请求：改了啥，为什么改 - 有权限的人审核 - checkin

度量与分析活动

GQM方法



1. 建立度量目标：自顶向下建立，我的目标是什么，建立不同的度量方式
2. 指定度量方式：度量的细节 - 代码行，前端后端不同效率
3. 收集保存
4. 分析

图上半部分是度量分析的规划准备工作，下半部分是实施

决策分析

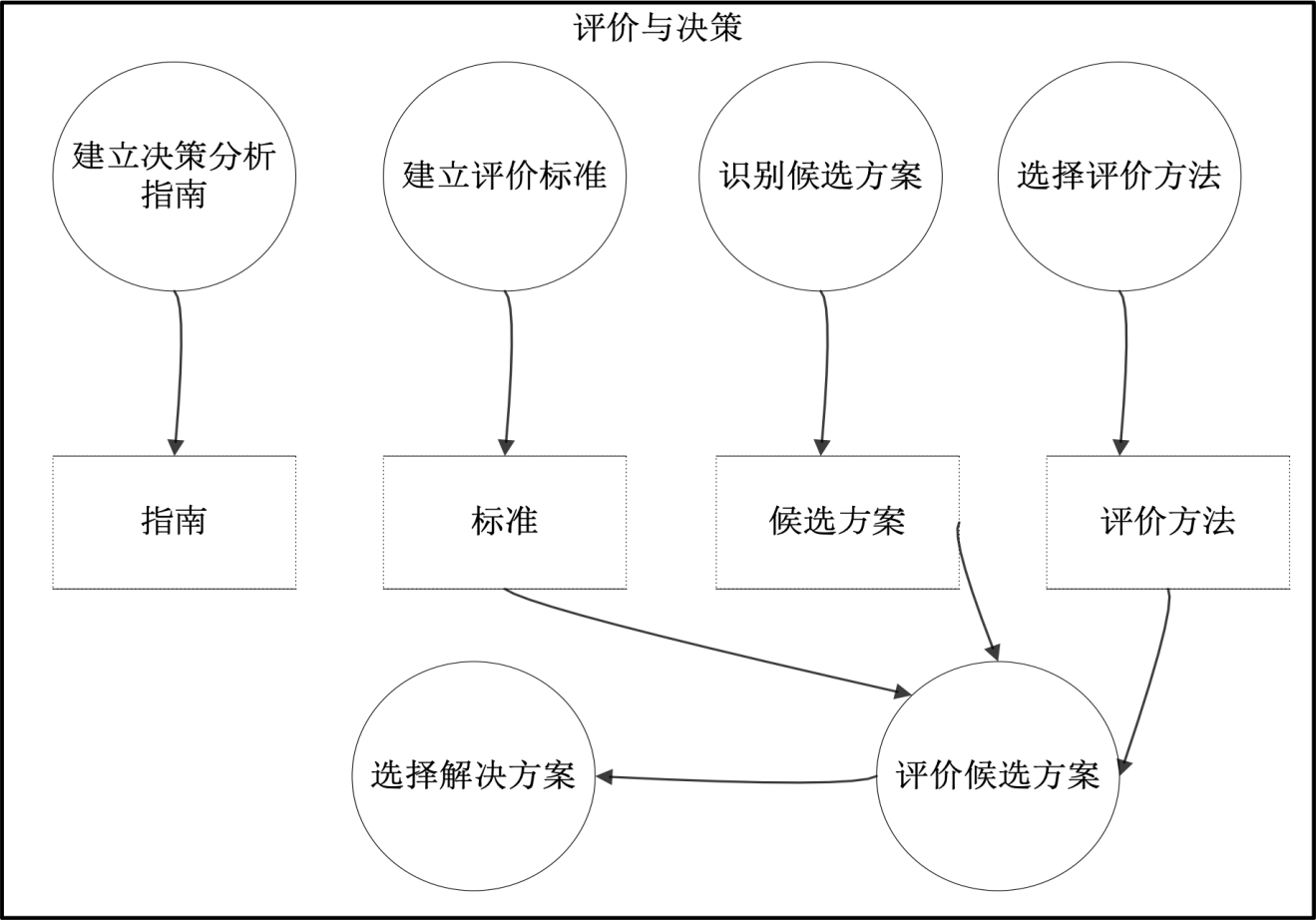
招标过程 - 每个小组代表一个解决方案

决策分析的意义：错误的决策往往会给项目带来灾难性后果。为了降低这种错误决策的风险，往往需要尽可能基于客观事实和正确的流程来开展决策与分析活动

困难：

一个正式评估过程往往包含下列的活动：

- 1. 建立评估备选方案的准则
- 2. 识别备选解决方案
- 3. 选择评估备选方案的方法
- 4. 使用已建立的准则与方法，评估备选解决方案
- 5. 依据评估准则，从备选方案中选择建议方案

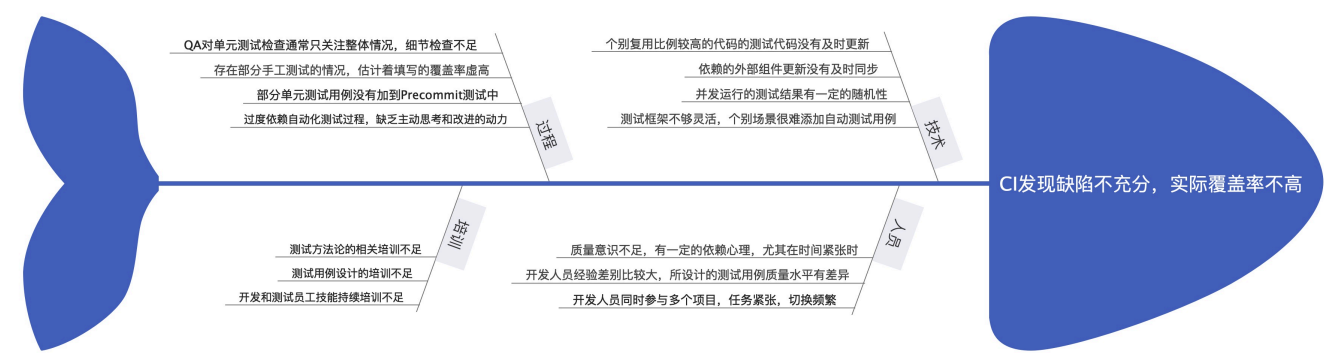


- 1. 决策分析指南：到达什么标准才需要决策
- 2. 建立评价标准

根因分析与解决方案

- 1. 避免类似错误反复发生
- 2. 一个正式根因分析过程往往包含下列的活动：
 - 1. 识别和选定问题
 - 2. 根因分析
 - 3. 建立改进的行动方案
 - 4. 实施改进，评估效果
- 3. 80%的问题集中在20%的部分上；怕内托图

根因分析典型示例



鱼骨图 - 根因分析主要手段

典型角度：技术角度，人员角度，培训角度，过程角度

08

定量管理仿真建模

先拆解，在控制每个关键子过程定量

确保每个最小的自变量都能满足要求

期末总结

过程线：

1. 什么是管理
2. 管理视角软工
3. 生命周期模型用于描述软件过程
4. PDCA IDEAL
5. CMM CMMI 指导过程管理
6. 软件危机
7. 三大阶段
8. 本质难题

项目管理线：

1. 激励方式
2. 自主团队
3. TSP/SCRUM
4. 估算
5. 项目计划
6. 定量计划（自顶向下）
7. 跟踪 - 挣值管理 - 最大优势：适应变更
8. 定量管理跟踪 自底向上

质量管理线：

1. 基本策略，缺陷管理替换质量管理 - 面向用户质量观
2. 个人评审 - 在编译之前评审why
3. 小组评审 - 九宫图
4. 质量管理指标
5. 质量journey
6. 设计：内部外部，动态静态 - 设计层次
7. 设计评审

工程技术线

1. 集成四种策略的特点
2. 验证和确认

考理解能力？题型和以前很不一样